

Revised Design with a Thread Pool

In this revised design:

- **Thread Pool:** We'll introduce a simple thread pool to handle auction tasks and bid processing tasks.
- **Task Submission:** Auction start, bidding, and result processing tasks will be submitted to the thread pool instead of spawning individual threads.
- **Synchronization:** We'll continue using `std::mutex`, `std::lock_guard`, and `std::condition_variable` to protect shared data and notify threads about auction end events.

Here's a basic thread pool implementation and how it would be integrated into our auction application.

Step 1: Implement a Simple Thread Pool

```
#include <vector>
#include <queue>
#include <thread>
#include <future>
#include <functional>
#include <condition_variable>

class ThreadPool {
public:
    ThreadPool(size_t num_threads) {
        for (size_t i = 0; i < num_threads; ++i) {
            workers.emplace_back([this] {
                while (true) {
                    std::function<void()> task;
                    {
                        std::unique_lock<std::mutex> lock(queue_mutex);
                        condition.wait(lock, [this] { return stop || !tasks.empty(); });
                        if (stop && tasks.empty()) return;
                        task = std::move(tasks.front());
                        tasks.pop();
                    }
                    task(); // Execute the task
                }
            });
        }
    }
};
```

```

}

template<class F, class... Args>
auto enqueue(F&& f, Args&&... args) -> std::future<typename std::result_of<F(Args...)>::type> {
    using return_type = typename std::result_of<F(Args...)>::type;
    auto task = std::make_shared<std::packaged_task<return_type()>>(std::bind(std::forward<F>
(f), std::forward<Args>(args)...));
    std::future<return_type> result = task->get_future();
    {
        std::unique_lock<std::mutex> lock(queue_mutex);
        if (stop) throw std::runtime_error("enqueue on stopped ThreadPool");
        tasks.emplace([task]() { (*task)(); });
    }
    condition.notify_one();
    return result;
}

~ThreadPool() {
    {
        std::unique_lock<std::mutex> lock(queue_mutex);
        stop = true;
    }
    condition.notify_all();
    for (std::thread &worker : workers) worker.join();
}

private:
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;
    std::mutex queue_mutex;
    std::condition_variable condition;
    bool stop = false;
};

```

Step 2: Modify the Auction Application to Use the Thread Pool

Now we'll refactor the auction application to submit auction and bid tasks to the thread pool.

```

#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <string>

```

```

#include <chrono>
#include <future>
#include <vector>

class Auction {
public:
    Auction(std::string item, int duration_seconds)
        : item_name(std::move(item)), duration(duration_seconds), highest_bid(0),
    auction_ended(false) {}

    // Function to place a bid on the item
    std::future<std::string> placeBid(int bid, const std::string& bidder, ThreadPool& thread_pool) {
        return thread_pool.enqueue([this, bid, bidder]() {
            std::lock_guard<std::mutex> lock(bid_mutex);
            if (auction_ended) {
                return "Auction for " + item_name + " has ended. No more bids allowed.";
            }
            if (bid > highest_bid) {
                highest_bid = bid;
                highest_bidder = bidder;
                std::string confirmation = "New highest bid for " + item_name + ": " +
std::to_string(bid) + " by " + bidder;
                std::cout << confirmation << std::endl;
                return confirmation;
            } else {
                std::string message = "Bid of " + std::to_string(bid) + " by " + bidder + " was too
low for " + item_name;
                std::cout << message << std::endl;
                return message;
            }
        });
    }

    // Function to run the auction with a time limit
    void startAuction(ThreadPool& thread_pool) {
        thread_pool.enqueue([this] {
            std::cout << "Starting auction for " << item_name << " lasting " << duration << "
seconds.\n";
            auto start_time = std::chrono::steady_clock::now();

            while (std::chrono::steady_clock::now() - start_time < std::chrono::seconds(duration)) {
                std::this_thread::sleep_for(std::chrono::milliseconds(100));
            }
        });
    }
};

```

```

        {
            std::lock_guard<std::mutex> lock(bid_mutex);
            auction_ended = true;
        }
        cv.notify_all();
        std::cout << "Auction for " << item_name << " has ended.\n";
    });
}

// Function to get the final result of the auction
void waitForAuctionEnd(ThreadPool& thread_pool) {
    thread_pool.enqueue([this] {
        std::unique_lock<std::mutex> lock(bid_mutex);
        cv.wait(lock, [this] { return auction_ended; });
        if (highest_bid > 0) {
            std::cout << "Auction result for " << item_name << ": Sold to " << highest_bidder
                << " for " << highest_bid << std::endl;
        } else {
            std::cout << "Auction result for " << item_name << ": No bids received.\n";
        }
    });
}

private:
    std::string item_name;
    int duration;
    int highest_bid;
    std::string highest_bidder;
    bool auction_ended;
    std::mutex bid_mutex;
    std::condition_variable cv;
};

// Simulate bidding by different users
void simulateBidding(Auction& auction, const std::string& bidder, int max_bid, ThreadPool&
thread_pool) {
    for (int bid = 10; bid <= max_bid; bid += 10) {
        std::future<std::string> result = auction.placeBid(bid, bidder, thread_pool);
        std::cout << "Confirmation for " << bidder << ": " << result.get() << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(150));
    }
}

int main() {

```

```

ThreadPool thread_pool(4); // Create a thread pool with 4 threads

Auction auction1("Painting", 5);
Auction auction2("Antique Vase", 7);

// Start auctions using the thread pool
auction1.startAuction(thread_pool);
auction2.startAuction(thread_pool);

// Simulate bidding using the thread pool
std::thread bidder1(simulateBidding, std::ref(auction1), "Alice", 100, std::ref(thread_pool));
std::thread bidder2(simulateBidding, std::ref(auction1), "Bob", 80, std::ref(thread_pool));
std::thread bidder3(simulateBidding, std::ref(auction2), "Charlie", 90, std::ref(thread_pool));
std::thread bidder4(simulateBidding, std::ref(auction2), "Dave", 120, std::ref(thread_pool));

// Wait for auctions to end and display results
auction1.waitForAuctionEnd(thread_pool);
auction2.waitForAuctionEnd(thread_pool);

// Join all threads
bidder1.join();
bidder2.join();
bidder3.join();
bidder4.join();

return 0;
}

```

Explanation of the Updated Code

1. Thread Pool:

- We've added a `ThreadPool` class that manages a fixed number of worker threads. It allows tasks to be enqueued and executed by any available thread in the pool.

2. Task Submission:

- The `startAuction`, `placeBid`, and `waitForAuctionEnd` functions now accept a `ThreadPool` reference and submit their work to the thread pool using `enqueue`.
- This keeps the number of threads manageable and allows the application to scale gracefully as we add more auctions or bidders.

3. Reduced Overhead:

- By using the thread pool, we avoid creating and destroying threads repeatedly for each bid or auction, reducing overhead and improving performance, especially with multiple auctions and bidders.